

Information Retrieval Assignment – Indian Language Search System

This assignment focuses on building an **Information Retrieval (IR) system for Indian languages** such as Telugu, Hindi, Tamil, Kannada, Bengali, Malayalam, etc. The goal is to understand the challenges of processing and searching multilingual Indian-language text.

Think of the project as building a **mini Google Search Engine for HINDI (or another Indian language)**.

Information Retrieval (IR) : Finding relevant information from a large collection of documents, similar to how Google searches webpages.

Main Features

1. Text Preprocessing & Corpus Analysis

- Tokenization of words in an Indian script
- Unicode normalization for consistent text representation
- Stopword removal
- Optional stemming/lemmatization
- Corpus statistics analysis using:
 - **Zipf's Law**
 - **Heaps' Law**
- Generation of vocabulary statistics and log-log plots

TERM	MEANING	EXAMPLE	REQUIREMENT
Tokenization	Splitting text into individual words.	भारत एक महान देश है। ↓ [भारत, एक, महान, देश, है]	Computers cannot search whole sentences efficiently. They need words separated first
Unicode	Converting different forms of the same text into one standard form.	कलम → कलम	The system treats both forms as the same word and doesn't miss results.
Stopword removal	Removing very common words that carry little meaning. "the", "is", "and"	भारत एक महान देश है ↓ भारत महान देश	Reduces unnecessary data and improves search efficiency.
Stemming	Reducing related words to root form	खेलना, खेलता, खेलती—>खेल	A search for खेल shud find all related words
Corpus Statistics	Analyzing the entire document collection.	500 Hindi Wikipedia articles.	For the project
Zipf's Law	A few words occur very frequently, while most words occur very rarely.	भारत → 500 times देश → 300 times	Shows word frequency patterns. helps in: Stopword selection, Index optimization
Heaps' Law	as the corpus grows, the vocabulary also grows but at a slower rate.	100 words -30 unique words 1000 words -150 unique words	Measures vocabulary growth as more documents are added.

Building the Search Engine

2. Inverted Index Construction & Compression

- Build an inverted index from scratch
- Store:
 - Dictionary of terms
 - Term Frequencies (TF)
 - Document Frequencies (DF)
- Support:
 - Boolean Queries (AND, OR, NOT)
 - Phrase Queries
- Implement positional indexing
- Serialize and deserialize indexes
- Index compression using:
 - Gap Encoding
 - Variable Byte Encoding or Gamma Encoding
- Measure compression ratio and storage savings

TERM	MEANING	EXAMPLE		REQUIREMENT
Inverted Index	A structure that stores which documents contain each word	भारत → [Doc1,Doc3,Doc5] देश → [Doc1, Doc2]		System jumps directly to relevant docs, instead checking each doc
Index Compression	Reduces index size	भारत → [1, 3, 5, 10] देश → [1, 2, 8]		Smaller storage size Faster loading,Faster search
Gap Encoding	stores the difference between the doc IDs instead of storing the actual doc IDs	N: [1000,1050,1150] G.E: [1000,50,50]		smaller numbers can be compressed more efficiently.
Phrase Queries	Searching for exact word sequences	"भारत देश" Matches: भारत देश NOT: देश भारत		Users often search for exact phrases.
Positional Indexing	stores <u>where a word occurs inside a document</u> , not just which document contains it.	भारत: {Doc1:[0],Doc2:[0]} देश : {Doc1:[3]} }		to support phrase searching and proximity queries.
Serialization	saving an index to a file for permanent storage.	Serialization: Start prog ↓ Load saved idx ↓ Ready	Start prog ↓ Build idx again ↓ Wait several minutes	Saving the index from RAM to a file. Much faster
Deserialization	loading a previously saved index back into memory use	Opening notebook and reading your notes again.		-

Making Search Smarter

3. Tolerant Retrieval

- **Wildcard Search**
 - K-gram indexing
 - Leading, trailing, and middle wildcards
 - Candidate filtering and verification
- **Transliteration Search**
 - Search using Latin script and retrieve results in native script
 - Support multiple transliteration mappings
 - Example: "bharat" → "भारत" / "भारत"

TERM	MEANING	EXAMPLE	REQUIREMENT
Wildcard Search	Searching with * as a placeholder.	भार* Matches: भारत, भारतीय, भारतीयता	Users may not know the complete word.
K-Gram Index	Breaking words into small character groups.	(K=2) भारत: भा, आर, रत	Makes wildcard searching fast
Transliteration	Converting English typing into Hindi script.	shiksha-> शिक्षा	Efficient use

Optional Bonus Features

- Sandhi/compound word handling
- Phonetic/confusable character matching
- Cross-script search (e.g., Hindi query on Telugu documents)

Cross-Script Search	Allowing searching across diff indian scripts	भारत -> భారత్
Phonetic Matching	Handles pronunciation-based spelling variations.	krishna Krushna; both referring to कृष्ण
Sandhi	a linguistic rule where sounds or letters change when words join together.	विद्या + आलय -> विद्यालय

Problem in Search Engines

A doc contains विद्यालय , but user searches विद्या	A normal search engine may not find the doc as विद्यालय ≠ विद्या
Idx both surface and decomposed components	राजकुमार -> राजकुमार, राज, कुमार

Dataset Requirements

- Minimum **500 documents**
- Minimum **100,000 words**
- Data can be collected from:
 - Wikipedia dumps
 - Wikisource
 - News articles
 - Literary collections

Technologies & Libraries Allowed

- Python 3
- NumPy, Pandas
- Matplotlib
- Unicode libraries
- Transliteration libraries

Libraries Not Allowed

- Elasticsearch
- Solr
- Whoosh
- PyLucene
- NLTK
- spaCy
- Gensim
- Scikit-learn
- Any pre-built IR system

Testing Requirements

The system must be tested for:

- Tokenization and normalization
- Stopword removal
- Zipf's and Heaps' law analysis
- Inverted index correctness
- Boolean and phrase queries
- Compression accuracy
- Wildcard retrieval
- Transliteration support

Deliverables

- Source code files for each module
- Unit tests
- Stopword list
- Configuration and result files
- README
- Report explaining implementation, performance, challenges, and results

Overall Objective

The assignment requires building a **complete Indian Language Information Retrieval System** from scratch that can preprocess text, create searchable indexes, perform efficient retrieval, support wildcard and transliteration queries, compress indexes, analyze corpus statistics, and evaluate search performance.